LECTURE SCRIPT

DDPG from Q-learning

1 Q-learning in general state–action spaces

In a general state–action space $\mathcal{S} \times \mathcal{A}$, we cannot store and learn a separate table entry for every pair (s, a). Learning a Q-function from finitely many observations is therefore a *function approximation problem*. We represent the action-value function by

$$Q_\phi : \mathcal{S} \times \mathcal{A} \longrightarrow \mathbb{R}, \qquad (s, a) \longmapsto Q_\phi(s, a), \tag{1}$$

where ϕ collects the parameters, for example the weights of a neural network.

The first idea: do the same as in DQN

For now, leave aside replay buffers, target networks, and the other DQN tricks. Given a nonterminal transition (s, a, s', r), form the usual Q-learning target and minimise the squared sampled TD error:

$$y = \mathrm{sg}\left(r + \gamma \max_{a' \in \mathcal{A}} Q_\phi(s', a')\right), \qquad \ell(\phi) = \frac{1}{2}\big(y - Q_\phi(s, a)\big)^2. \tag{2}$$

Here sg denotes stop-gradient: the target is held fixed when differentiating. A stochastic gradient descent step gives the familiar *semi-gradient* update

$$\phi \leftarrow \phi + \alpha_\phi \big(y - Q_\phi(s, a)\big) \nabla_\phi Q_\phi(s, a). \tag{3}$$

For a manageable finite action space, we can evaluate every action and take the largest value. This settles action selection and target computation, just as in DQN; the usual function approximation and stability issues remain.

The maximum appears in two places

With continuous actions, we cannot output one value per possible action (e.g. via a network). Taking both s and a as input solves the representation problem, but we still have a continuous optimisation problem:

1. **Learning:** compute $\max_{a'} Q_\phi(s', a')$ in the TD target.
2. **Exploration:** find an action in $\arg\max_a Q_\phi(s, a)$ for building an exploratory behaviour policy.

> **Does a maximiser even exist?**
> Before asking how to compute an arg max, we need it to exist. We will ignore this problem and if in doubt simply assume $\mathcal{A}$ is nonempty and compact and $a \mapsto Q_\phi(s, a)$ is continuous for every s.

Learning the maximiser

The desired output of the optimisation problem is itself a function mapping to a continuous action space: $s \mapsto \arg\max_a Q_\phi(s, a)$. Thus, determining it is another function approximation problem. Introduce

$$\mu_\theta : \mathcal{S} \longrightarrow \mathcal{A}, \qquad \mu_\theta(s) \approx \arg\max_{a \in \mathcal{A}} Q_\phi(s, a). \tag{4}$$

This function approximator produces an action directly and we can plug it into Q_ϕ to calculate the maximum in the TD target.

How do we train the maximiser?

We do not have labelled examples of optimal actions. Instead, we use the fact that μ_θ should approximate the optimal action according to Q_ϕ for each state. Holding ϕ fixed, choose a training-state distribution d_D and maximise

$$J_D(\theta;\phi) = \mathbb{E}_{s\sim d_D}\left[Q_\phi(s,\mu_\theta(s))\right]. \tag{5}$$

If the actor and critic are differentiable, the chain rule gives

$$\begin{aligned}\nabla_\theta J_D(\theta;\phi) &= \mathbb{E}_{s\sim d_D}\left[D_\theta\mu_\theta(s)^\top \nabla_a Q_\phi(s,a)\big|_{a=\mu_\theta(s)}\right],\\ \theta &\leftarrow \theta + \alpha_\theta \widehat{\nabla}_\theta J_D(\theta;\phi).\end{aligned} \tag{6}$$

Here $D_\theta\mu_\theta(s)$ is the action-by-parameter Jacobian. Backpropagation passes through the critic's action input into the actor; the critic parameters are fixed in this step. A few such steps improve the proposed actions, but need not find global maximisers.

Which states should train the maximiser?

Use the same data distribution as for Q-learning: this is where the learned maximiser is needed, rather than an arbitrary distribution over $\mathcal{S}$. More precisely, the bootstrap uses μ_θ at *successor states* s', whereas the Q-network is fitted at current states s.

For trajectory data, s' at one step is the current state at the next. The two collections therefore differ only at rollout or episode boundaries; under a stationary sampling regime their distributions coincide, and for long rollouts boundary effects are small. We usually train both networks on the same sampled current states and call their distribution d_D. If boundary effects matter, we can train the maximiser on the sampled s' directly. The Markov property alone does not imply equality of the two distributions.

Comparison with deterministic policy gradients (DPG)

The gradient in (6) has the same form as DPG. The DPG theorem starts from expected return, uses Q^{μ_θ}, and weights states by the target policy's discounted visitation distribution. A deterministic policy does not itself explore. Moving to an exploratory behaviour policy and replay changes that state distribution, so the uncorrected on-policy theorem no longer identifies the sampled update with the exact return gradient.

> **The Q-learning view gives the objective directly**
>
> Here d_D is deliberately the training-data distribution and Q_ϕ is fixed during actor optimisation. Equation (6) is the gradient of precisely the objective we want: make the proposed actions greedy where we use them. Starting from DPG instead requires an off-policy approximation or correction before using this replay-weighted update.

How do we explore?

The maximiser μ_θ is deterministic, but the policy used to collect data need not be. Q-learning is off-policy, so we can learn from an exploratory behaviour policy while using μ_θ in the bootstrap.

If actions have meaningful geometry, nearby actions are a natural way to explore locally. For $\mathcal{A} = [a_{\min}, a_{\max}] \subset \mathbb{R}^m$, a common choice is Gaussian noise around the actor output:

$$\xi_t \sim \mathcal{N}(0,\sigma^2 I), \qquad a_t = \mathrm{clip}(\mu_\theta(s_t) + \xi_t, a_{\min}, a_{\max}). \tag{7}$$

Before clipping, the action has distribution $\mathcal{N}(\mu_\theta(s_t),\sigma^2 I)$. Clipping is componentwise and ensures feasibility. More general action constraints require a suitable feasible perturbation or projection.

If local distance has no useful interpretation, we can instead imitate ε-greedy exploration:

$$a_t = \begin{cases} \mu_\theta(s_t), & \text{with probability } 1-\varepsilon, \\ U_t, \quad U_t \sim \mathrm{Unif}(\mathcal{A}), & \text{with probability } \varepsilon. \end{cases} \tag{8}$$

This uses a uniform distribution when one is available, for example on a bounded action set of positive volume. Otherwise choose an appropriate reference exploration distribution. The behaviour policy can also be chosen independently of μ_θ.

Q-learning or actor-critic?

Replace the maximisation in the target by the learned maximiser:

$$y = r + \gamma Q_\phi(s', \mu_\theta(s')). \tag{9}$$

If $\mu_\theta(s') \in \arg\max_{a'} Q_\phi(s', a')$, this is exactly the Q-learning target. We have implemented Q-learning in a continuous action space by learning how to perform its maximisation.

If the actor only takes partial improvement steps, the target instead uses the action of the *current policy*. With that policy fixed, the critic's backup is a policy evaluation backup; the actor then improves its actions using the critic. These are the two parts of an *actor-critic* algorithm. This is also why DDPG (Deep Deterministic Policy Gradient) is normally classified as an off-policy actor-critic method.

	Network	**Role**
Actor	$\mu_\theta(s)$	Propose actions; perform policy improvement.
Critic	$Q_\phi(s,a)$	Evaluate actions; provide the actor's learning signal.

The parallel with policy iteration and value iteration

Policy iteration alternates policy evaluation and policy improvement. Value iteration combines greedy improvement with just *one* evaluation backup at a time. In Q-function notation, if μ is greedy with respect to the current Q, then

$$(T^\mu Q)(s,a) = \mathbb{E}[r + \gamma Q(s', \mu(s')) \mid s, a] = \mathbb{E}\left[r + \gamma \max_{a'} Q(s', a') \mid s, a\right] = (T^* Q)(s,a). \tag{10}$$

Thus, exact greedification makes the one-step actor-critic backup a Q-learning backup.

2 DDPG in practice

We now add the basic DQN machinery: a replay buffer D, minibatch updates, and slowly changing target networks. Keep a target critic Q_{ϕ^-} *and* a target actor μ_{θ^-} so that the entire bootstrap is fixed between target updates. Store transitions (s_i, a_i, s_i', r_i). The target and the two minibatch objectives are

$$y_i = r_i + \gamma Q_{\phi^-}(s_i', \mu_{\theta^-}(s_i')), \tag{11}$$

$$L_B(\phi) = \frac{1}{2|B|}\sum_{i\in B}\big(Q_\phi(s_i, a_i) - y_i\big)^2, \qquad J_B(\theta;\phi) = \frac{1}{|B|}\sum_{i\in B} Q_\phi(s_i, \mu_\theta(s_i)). \tag{12}$$

The critic minimises L_B; the actor maximises J_B. All y_i are detached from differentiation. In the actor step, the critic is fixed but its derivative with respect to the *action* is used.

Algorithm 1: DDPG from Q-learning

Initialize replay buffer D, critic parameters ϕ, actor parameters θ, and target networks $\phi^- \leftarrow \phi, \theta^- \leftarrow \theta$.
  Choose buffer capacity $|D|$, batch size $|B|$, step sizes $\alpha_\phi, \alpha_\theta$, and target update interval N.
repeat
  Collect and pre-process a transition (s, a, s', r, d) using a behaviour policy and add it to D.
  Sample a minibatch B of $|B|$ transitions uniformly from D.
  Compute the targets $y_i = r_i + \gamma Q_{\phi^-}(s_i', \mu_{\theta^-}(s_i'))$.
  Take an optimiser step each using
  $\nabla_\phi L_B = \frac{1}{|B|}\sum_{i\in B}(Q_\phi(s_i, a_i) - y_i)\nabla_\phi Q_\phi(s_i, a_i)$ and $\nabla_\theta J_B = \frac{1}{|B|}\sum_{i\in B} D_\theta \mu_\theta(s_i)^\top$.
  Every N steps, update the target networks: $\phi^- \leftarrow \phi, \theta^- \leftarrow \theta$.
until the training budget is exhausted.

A few implementation choices

The DQN toolbox remains useful. Replay, minibatches, and target networks are already present above. Further options include the **Huber loss** for the critic, observation and consistent target/output **normalisation**, and rescaling the actor's output to the action bounds. Rainbow-style components such as prioritised replay (with sampling corrections), multi-step returns (with appropriate off-policy treatment), noisy networks, and distributional critics can also be adapted. Components tied to enumerating discrete actions, such as the usual dueling head, need a continuous-action formulation. Differing from DQN and using the AC structure, two networks can use different minibatches, batch sizes, replay buffers, capacities, or numbers of updates. Sharing these parameters, however, is a convenient default.

Convexity of the maximum gives overestimation for Q-learning. The same issue persists when the maximum is well-defined over continuous actions. The actor moves towards actions with large estimated Q-values. A partial policy improvement step is a kind of *soft max*, in the sense of incomplete greedification, and can still lead to overestimation due to the same convexity issue. With many similarly valued actions and comparable, imperfectly correlated errors, more candidate actions create more opportunities to select an overestimate. Thus DDPG can show *more*, rather than less, overestimation than DQN: this largely reflects the rich continuous action spaces to which we apply it. Given this persisting overestimation, the wide array of overestimation reduction methods available to DQN-style algorithms can be immediately adapted to DDPG.

Instead of periodically copying all parameters, we can use exponential averaging:

$$\phi^- \leftarrow (1-\tau_\phi)\phi^- + \tau_\phi\phi, \qquad \theta^- \leftarrow (1-\tau_\theta)\theta^- + \tau_\theta\theta. \tag{13}$$

These *soft target updates* are regularly used in DQN as well as DDPG instead of *hard target updates* after some fixed number of steps N. An update from k steps ago has weight $\tau(1-\tau)^k$, so the geometric memory timescale is $N_{\text{eff}} \approx \frac{1}{\tau}$. Soft averaging smooths parameter changes, whereas hard updates keep targets fixed and then replace them.